Vision & Constraints 🔒 LOCKED

Locked artifact. These principles govern every design decision in the DRCS engine.

The Vision

User Goal: “I want a tool I can use myself to make content out of any idea.”

Delivery: A natural language prompt → real media file path + caption. Either reused from a library or freshly generated.

Architecture: One engine with eight configurable components (C1–C8) serving as gates. Single canonical codebase with deployment-specific behavior via configuration only.

Hard Constraints

1. Single Canonical Codebase

One repo, one build. Deployment-specific behavior is achieved **entirely through configuration** — tenant config, category schemas, asset registries. Zero hardcoded tenant IDs in gate logic.

2. Multi-Tenant Architecture with Strict Isolation

Every database query, every gate call, every audit trail is tenant-scoped. A tenant’s data is invisible to every other tenant. Enforced at the persistence layer via `tenant_id` filters on every read/write.

3. Fail Closed on Any Log Integrity Issue

C3 (Governor) checks usage-log integrity before allowing deployment. If the log cannot be verified (missing records, corrupted timestamps), the gate **fails closed** (blocks deployment) rather than risking a silent violation of the frequency cap.

4. C3 Locked Parameters

- `max_count = 3`
- `rolling_window = 30 days`
- `counting_unit = per deployment instance`

Not configurable per request. Enforcement is uniform. Overrides require explicit tenant-config policy flags.

5. Strict 3-Step Check Before New Generation

C4 (Resolution) **must** attempt these steps in order:
1. **Existing-as-is** — exact caption match in the resolved category
2. **Recaption** — reuse an existing asset with a new caption
3. **Escalate to C5** — only if steps 1 & 2 yield no usable content

Never generate fresh content if existing content can be reused or recaptioned. This is the “caption-first” contract.

6. No Date/Calendar Selection Path

C2 (Situational Bank) has **zero** date/time/weekday lookup logic. Category resolution is driven exclusively by real-time condition signals (explicit `category_id` or `situation` label). A static source scan test confirms no `Date()`, `getDay()`, or calendar APIs in compiled C2 output.

Rationale: Protected categories like “Friday” are resolved because an upstream signal reports the Friday situation, never because the engine checked the calendar. Cadence is preserved externally, not internally.

7. One Gate = One Branch = One PR Targeting Master

Git workflow: Every gate was built on its own feature branch (`feat/c1-lock`, `feat/c2-bank`, etc.) and merged to `master` via individual PRs. No stacking branches.

Exception: When integration required combining C3/C4/C5/orchestrator changes into a single coordinated PR (`feat/orchestrator-and-ui`), the branches were explicitly integrated and the PR targeted `master` directly.

Architecture Decisions

Decision: Add an Orchestrator Layer

Problem: Raw gates alone were “gears with no machine” — they returned decisions but didn’t retrieve content or handle the LLM.
Solution: Built `src/orchestrator/index.ts` which runs the gate sequence (C6 → C2 → C4 → C5 → C3) with short-circuiting, audit trail, and content retrieval. The orchestrator is the “machine.”

Decision: Add C5 (Misalignment Protocol)

Problem: C4 escalation had nowhere to go — the pipeline dead-ended.
Solution: Built C5 to handle LLM-based fresh caption generation. Now escalation proceeds instead of stopping.

Decision: Add Natural Language `evaluatePrompt` Entry Point

Problem: The multi-field structured form was unusable for the intended user (“make content out of any idea”).
Solution: Added `evaluatePrompt(tenant_id, prompt)` — one text input, LLM extracts intent and maps it to a category name from the tenant’s taxonomy, engine handles the rest.

Decision: LLM Prompt Redesign for Category Mapping

Problem: Original `extractPromptFields` let the LLM generate a free-form “situation” description (e.g., “Celebrating 10k followers”). C2 does exact name matching against category names (e.g., “Victory”). The two never connected.
Solution: Changed `extractPromptFields` to:
1. Load the tenant’s actual category names from the DB (`categorySchema.listCategories`)
2. Pass those names to the LLM as a closed list
3. Ask the LLM to pick the single best-matching category name verbatim
4. Return that exact name as `situation` so C2 can match it

Now the LLM maps user intent → real category name → C2 resolves it → content returned.

Rejected Alternatives

Rejected: Hardcoded Tenant Logic in Gates

Why: Violates single-codebase principle. Adding a second tenant would require code changes in every gate.
Chosen: Tenant-scoped persistence. Gates load config/categories/assets at runtime.

Rejected: Seed Demo Data as Canonical

Why: The `demo` seed (in `src/seeds/demo.ts`) assigns arbitrary clips to categories for UI testing. It's illustrative only, not the real Zilly mapping.
Chosen: Treat `demo` as non-canonical. The real Zilly taxonomy/asset mapping is TBD and will be seeded separately.

Rejected: Stacking Feature Branches

Why: Creates merge-conflict hell and obscures which changes belong to which gate.
Chosen: One gate per branch, each PR targets `master` directly. Exception: orchestrator integration PR combined C3/C4/C5/orchestrator because they were interdependent.

What This Document Governs

Every design decision, every gate contract, every test acceptance criterion flows from these locked principles. If a proposed change violates any constraint here, it is rejected unless this document is explicitly reopened and the constraint is revised.

Locked by: Project owner
Immutable unless: Explicitly reopened for revision